

RF19 — Definição da Arquitetura de Dados: ETL, ELT e Arquitetura Híbrida

Programa: FIC DEV IA — Programador de Sistemas com IA
Módulo: Fundamentos de Dados para IA
Projeto: Pipeline Governado, Escalável e Seguro de Conteúdos Educacionais

1. Visão Geral da Arquitetura Adotada

O projeto evoluiu da abordagem pontual do Desafio 1 para uma **Arquitetura Híbrida Moderna (Medallion Architecture)** combinando os paradigmas de **ETL (Extract, Transform, Load)** e **ELT (Extract, Load, Transform)** ao longo das diferentes camadas de maturidade do dado.

FLUXO INTEGRADO DA ARQUITETURA HÍBRIDA			
Macro-Etapa	Camadas Envolvidas	Paradigma Dominante	Motor de Execução Tecn
1. Ingestão	Fontes → Bronze	EL (Extract & Load)	Apache Hop (Pipelines
2. Padronização	Bronze → Silver	ETL / Limpeza	Apache Hop (Validação
3. Agregação Distribuída	Silver → Parquet → Camada Gold	Processamento Escalável (Distributed)	Apache Beam (DirectRun com particionamento
4. Consumo Analítico	Gold → Superset	ELT / In-Database Virtual Datasets	PostgreSQL Engine (Vie com cache no Apache Su

2. Diagrama de Fluxo Ponta a Ponta (Arquitetura Medallion)

```
flowchart TD
    subgraph FONTES["1. Fontes de Dados (Desafio 1)"]
        F1["CSV Catálogo"]
        F2["JSON Interações"]
        F3["JSON Comentários"]
    end

    subgraph BRONZE["2. Camada Bronze (Raw Ingestion)"]
        B1["bronze.catalogo"]
        B2["bronze.interacoes"]
    end
```

```

    B3[("bronze.comentarios")]
end

subgraph TRATAMENTO["3. Orquestração e Validação"]
    HOP["Apache Hop Workflows"]
    Q["quarentena.interacoes\n(Registros Inválidos / Erros)"]
    GATE{"Quality Gate\n(executar_testes.py)"}
end

subgraph SILVER["4. Camada Silver (Standardized & Parquet)"]
    S1[("silver.catalogo")]
    S2[("silver.interacoes")]
    P1["Parquet Particionado\n(ano_mes=YYYY-MM)"]
end

subgraph DISTRIBUIDO["5. Processamento Distribuído"]
    BEAM["Apache Beam Pipeline\n(pipeline.py)"]
end

subgraph GOLD["6. Camada Gold (Curated & Analytical)"]
    G1[("gold.engajamento_conteudo")]
    G2[("gold.engajamento_conteudo_mensal")]
    V1["Visões: vw_resumo_categoria\nvw_conteudos_atencao"]
end

subgraph GOVERNANCA["7. Governança e Metadados"]
    OM["OpenMetadata\n(Catálogo, Linhagem e Glossário)"]
end

subgraph CONSUMO["8. Camada de Consumo Executivo"]
    LAB["SQL Lab Datasets Virtuais"]
    DASH["Dashboard Superset\n(Cross-Filtering e Alertas)"]
end

FONTES →|Ingestão Fiel sem Perda| BRONZE
BRONZE → HOP
HOP →|Registros Reprovados| Q
HOP →|Registros Validados| SILVER
SILVER → GATE
GATE →|Aprovado 100%| P1
P1 → BEAM
BEAM → GOLD
GOLD → OM
GOLD → LAB
LAB → DASH

```

3. Classificação e Identificação das Fases (Extração, Transformação e Carga)

Fase no Pipeline	Onde Ocorre	Paradigma	Descrição da Operação
E (Extração)	Leitura das fontes (arquivos CSV, JSON e conexões PostgreSQL)	E	Leitura em streaming/chunks preservando dados brutos originais.
L1 (Carga Bronze)	Escrita em <code>bronze.*</code>	L	Carga imutável com metadados de auditoria (<code>origem_arquivo</code> , <code>data_hora_ingestao</code> , <code>execucao_id</code>).
T1 (Transformação Silver)	Apache Hop Pipelines	T	Tipagem, tratamento de nulos, deduplicação por chave de negócio e separação de anomalias na Quarentena.
L2 (Carga Silver)	Escrita em <code>silver.*</code> e arquivos Parquet	L	Armazenamento colunar estruturado e otimizado para analítica distribuída.
T2 (Transformação Distribuída)	Apache Beam	T	Agregações de engajamento, percentual de conclusão e métricas temporais por janela de negócio.
L3 (Carga Gold)	Escrita em <code>gold.*</code>	L	Carga das tabelas consolidadas preparadas para consultas analíticas.
T3 (Transformação In-Database)	PostgreSQL Views / SQL Lab	T	Enriquecimento sob demanda com expressões condicionais (<code>CASE WHEN</code>) e junções analíticas consumidas pelo Superset.

4. Justificativa da Escolha Arquitetural

- A escolha por uma **arquitetura híbrida** fundamenta-se nos seguintes pilares de engenharia:
- Custo Computacional Otimizado:** Transformações volumosas de agregação temporal são delegadas ao Apache Beam sobre arquivos Parquet colunares, poupando a CPU do banco de dados relacional durante cálculos pesados.
 - Governança e Rastreabilidade Integral (Data Lineage):** Ao manter a camada Bronze imutável, qualquer necessidade de auditoria ou investigação de incidentes pode ser reproduzida a partir da origem sem risco de corrupção.
 - Desempenho de Consumo (Low Latency):** O Apache Superset não consulta tabelas transacionais nem a camada Bronze. Todas as consultas operam na Camada Gold pré-agregada e nos datasets virtuais otimizados.
 - Resiliência e Reprocessamento (Data Recovery):** Em caso de falha em regras de negócio, os dados isolados na Quarentena podem ser reprocessados pelo Apache Hop sem necessidade de reingerir o arquivo bruto original.

5. Limitações da Solução Anterior (Desafio 1) vs Ganhos do Desafio 2

Aspecto Técnico	Desafio 1 (Scripts Isolados)	Desafio 2 (Arquit
Orquestração	Scripts Python soltos executados manualmente em ordem arbitrária.	Workflows formais tratamento de err
Gestão de Falhas	O script quebrava totalmente ao encontrar um dado inconsistente.	Registros inválid Quarentena, permi
Escalabilidade	Processamento monolítico na memória da máquina do desenvolvedor.	Processamento par Apache Beam distr
Governança e Linhagem	Metadados e regras dispersos em comentários ou desconhecidos.	Catálogo centrali com termos de neg
Consumo Analítico	Consultas manuais pontuais em SQL.	Dashboard interat Cross-Filtering e